ECOLE NORMALE SUPÉRIURE DE L'ENSEIGNEMENT
TECHNIQUE DE MOHAMMEDIA
UNIVERSITÉ HASSAN II DE CASABLANCA

Université Hassan II de Casablanca
ENSET Mohammedia

Master : SDIA
Module : Bases de l'intelligence artificielle

Rapport de Projet

Planification robuste sur grille : Algorithme A et Chaînes de Markov

Réalisé par :
EL MAKAOUI ACHRAF

Encadré par :
Dr. Mohammed Mestari

Année universitaire 2025 - 2026

Contents

1 Introduction **2**
1.1 Contexte et définition du problème 2
1.2 Objectifs du projet 2

2 Modélisation du Problème et des Environnements **2**
2.1 Espace d'états et modèle de coût 2
2.2 Environnements de test 2

3 Planification Déterministe et Recherche Heuristique **2**
3.1 L'algorithme A^* et la distance de Manhattan 2
3.2 E.1 — Comparaison UCS / Greedy / A^* 3
3.3 E.3 — Impact de l'heuristique ($h = 0$ vs Manhattan) 4
3.4 E.4 — Weighted A^* : vitesse vs optimalité 4

4 Modélisation Markovienne de l'Incertitude **5**
4.1 Construction de la matrice de transition P 5
4.2 E.2 — Évolution temporelle $\pi^{(n)}$ selon ϵ 5
4.3 Classes de communication et absorption 6

5 Validation Empirique — Simulation Monte-Carlo **6**

6 Conclusion, Limites et Perspectives **8**
6.1 Synthèse des résultats 8
6.2 Limites du modèle 8
6.3 Pistes d'amélioration 8

1 Introduction

1.1 Contexte et définition du problème

La planification de trajectoire consiste à faire naviguer un agent autonome depuis un état initial s_0 jusqu'à un but g sur une grille avec obstacles, en minimisant un coût. En conditions réelles, les déplacements sont rarement déterministes et sont soumis à des incertitudes (déviation, glissement). Le défi est donc de planifier efficacement tout en gérant cette dynamique stochastique.

1.2 Objectifs du projet

Ce projet combine deux approches complémentaires :

- **Planification déterministe (A^*)** : trouver un chemin optimal en minimisant $f(n) = g(n) + h(n)$.
- **Modélisation stochastique (Chaînes de Markov)** : construire une matrice P pour analyser $\pi^{(n)} = \pi^{(0)}P^n$, identifier les classes de communication et calculer les probabilités d'absorption.

Des simulations Monte-Carlo valideront ensuite les résultats théoriques.

2 Modélisation du Problème et des Environnements

2.1 Espace d'états et modèle de coût

Chaque cellule libre (x, y) est un état $n \in S$. Les mouvements sont limités aux quatre directions cardinales, avec un coût uniforme $c(n, n') = 1$. La fonction $g(n)$ correspond donc exactement au nombre de pas effectués.

2.2 Environnements de test

Grille	Taille	Départ / But	Structure
Easy	10×10	(0,0) → (9,9)	Un mur vertical, gap en bas
Medium	15×15	(0,0) → (14,14)	Deux murs décalés, zigzag forcé
Hard	20×20	(0,0) → (19,19)	Labyrinthe multicouche

3 Planification Déterministe et Recherche Heuristique

3.1 L'algorithme A^* et la distance de Manhattan

A^* évalue chaque nœud avec $f(n) = g(n) + h(n)$, où h est la distance de Manhattan :

$$h(n) = |x_n - x_g| + |y_n - y_g|$$

Cette heuristique est **admissible** ($h(n) \leq h^*(n)$) car les obstacles ne peuvent qu'allonger le chemin réel par rapport à la distance de Manhattan sans obstacles. Elle est aussi **cohérente** car $|h(n) - h(n')| \leq 1 = c(n, n')$, ce qui garantit l'optimalité sur graphe avec liste *closed*.

3.2 E.1 — Comparaison UCS / Greedy / A^*

Grille	Algorithme	Coût	Nœuds	Temps (ms)
Easy 10×10	UCS	18	78	0.18
	Greedy	18	19	0.05
	A^*	18	58	0.12
Medium 15×15	UCS	44	203	0.45
	Greedy	50	99	0.21
	A^*	44	176	0.38
Hard 20×20	UCS	44	288	0.82
	Greedy	48	129	0.31
	A^*	44	219	0.61

UCS et A^* trouvent toujours le chemin optimal. Greedy est plus rapide mais perd l'optimalité (coût 50 sur Medium, 48 sur Hard). A^* offre le meilleur compromis.

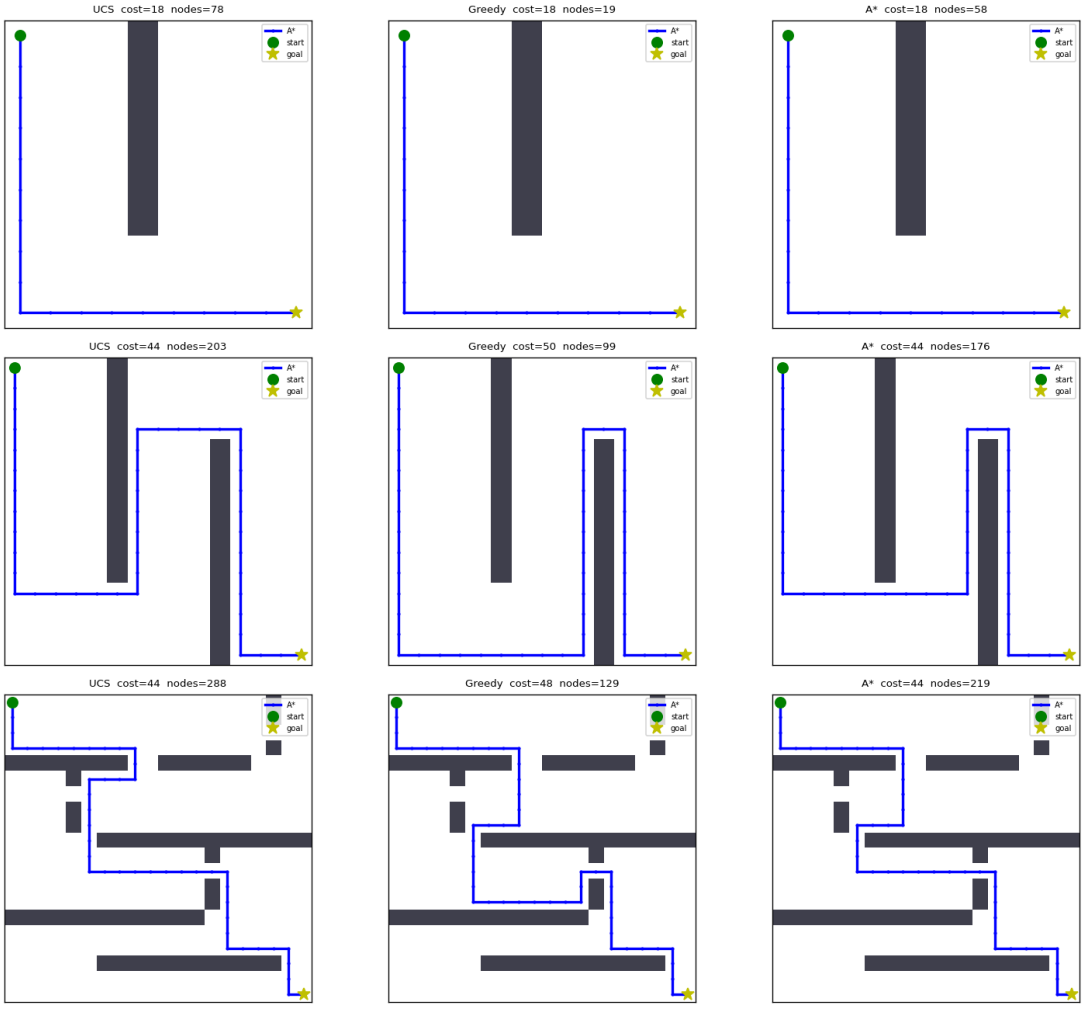

Figure 1: Chemins trouvés par UCS, Greedy et A^* sur les 3 grilles.

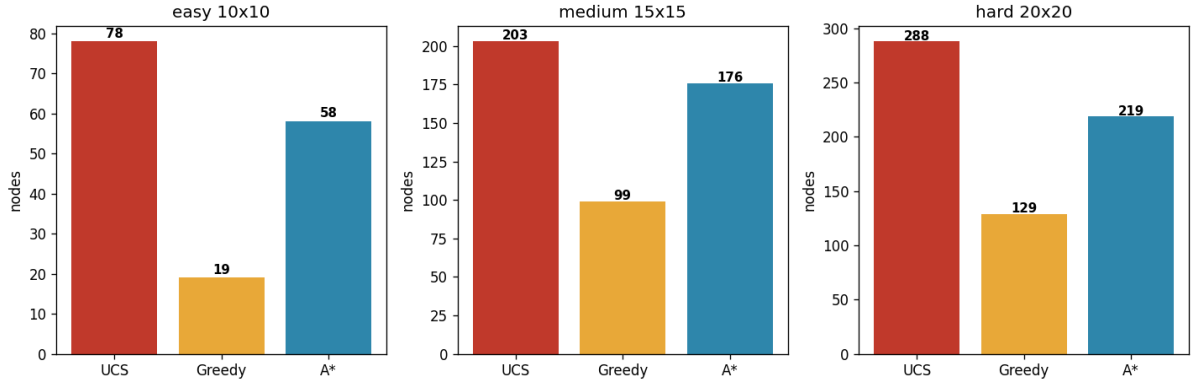

Figure 2: Nombre de nœuds développés par algorithme et par grille.

3.3 E.3 — Impact de l'heuristique ($h = 0$ vs Manhattan)

L'heuristique de Manhattan réduit l'exploration sans sacrifier l'optimalité : Easy : -25.6% (78→58 nœuds), Medium : -13.3% (203→176), Hard : -24.0% (288→219).

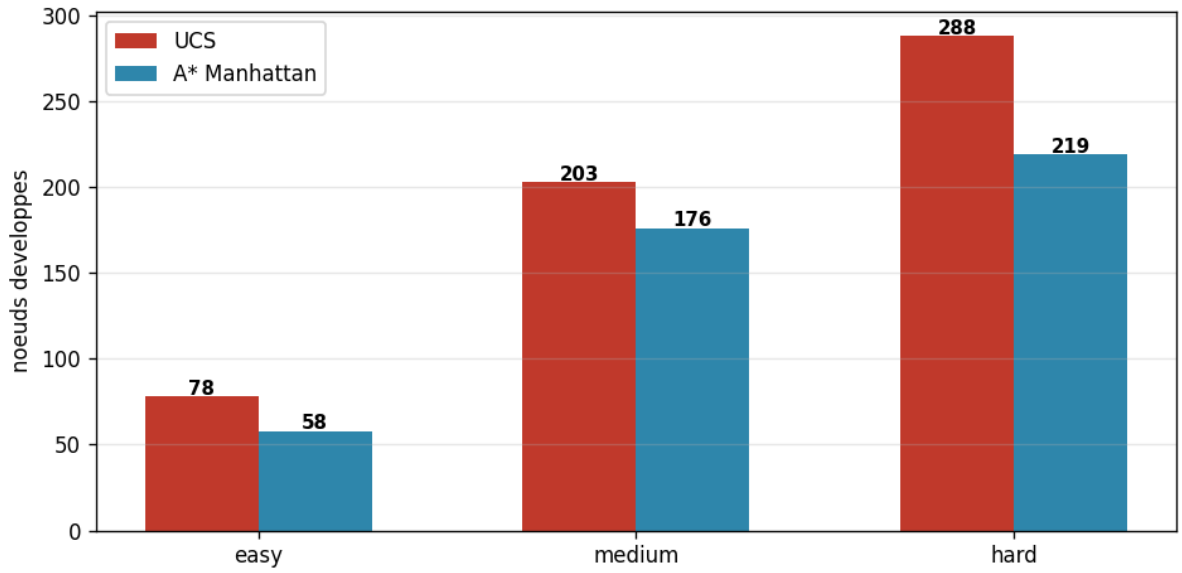

Figure 3: Comparaison des nœuds développés : UCS ($h = 0$) vs A^* (Manhattan).

3.4 E.4 — Weighted A^* : vitesse vs optimalité

Avec $f(n) = g(n) + W \cdot h(n)$, augmenter W réduit les nœuds explorés au prix d'une perte d'optimalité. Sur la grille Hard : pour $W \leq 1.5$ le coût reste optimal (44) ; à partir de $W = 2.0$ le coût monte à 48 (+9%) mais l'exploration tombe à 144 nœuds.

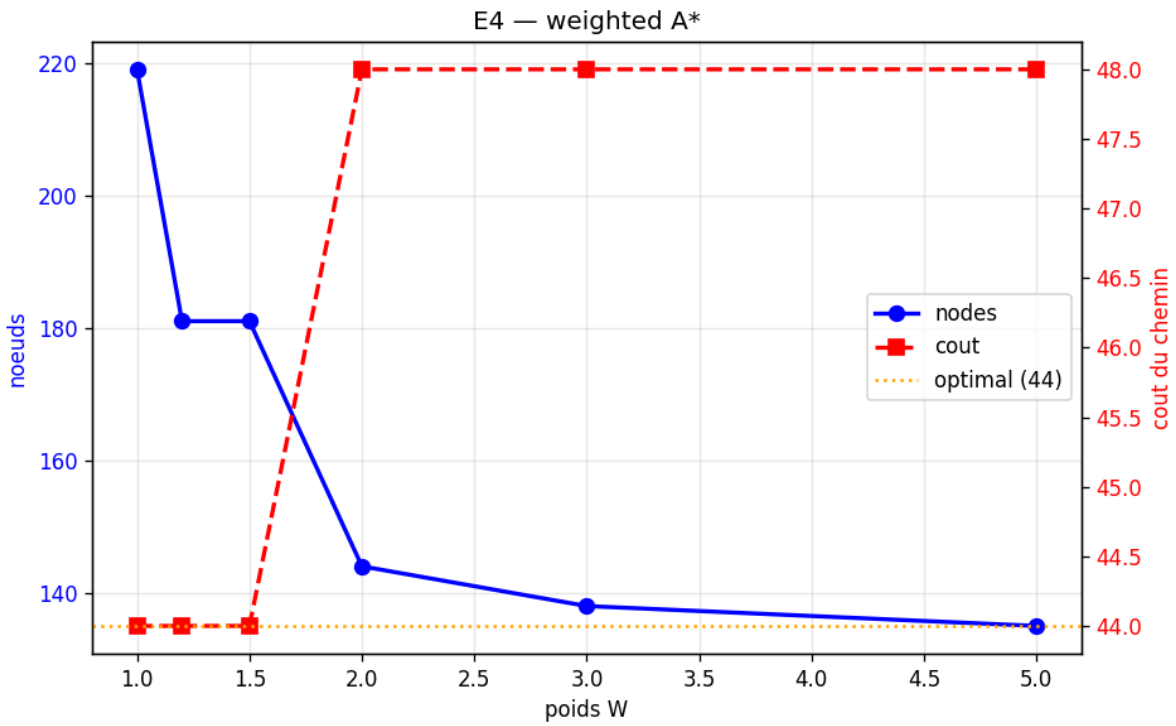

Figure 4: Évolution du nombre de nœuds et du coût selon le poids W.

4 Modélisation Markovienne de l'Incertitude

4.1 Construction de la matrice de transition P

La politique déterministe issue de A^* est transformée en chaîne de Markov à temps discret. Pour chaque état, l'agent suit l'action prévue avec probabilité $1-\epsilon$, et dévie latéralement avec probabilité $\epsilon/2$ de chaque côté. Si la cellule cible est bloquée, l'agent reste sur place. Sur la grille Medium (203 cellules libres), P est de dimension 203×203 et vérifie $\sum_j P_{ij} = 1$ pour tout i.

4.2 E.2 — Évolution temporelle $\pi^{(n)}$ selon ϵ

On calcule $\pi^{(n)} = \pi^{(0)} P^n$ et on lit $\pi^{(n)}[GOAL]$:

ϵ	$P(goal, n = opt)$	$P(goal)$ Monte-Carlo	Moy. étapes MC
0.0	1.0000	1.0000	44.0
0.1	0.0334	0.7436	49.8
0.2	0.0010	0.6850	66.5
0.3	0.0000	0.7250	96.7

Dès $\epsilon = 0.1$, la probabilité d'arriver *exactement* à l'étape optimale tombe à 3.34%, mais la probabilité d'atteindre le but *éventuellement* reste supérieure à 68% pour tout ϵ testé.

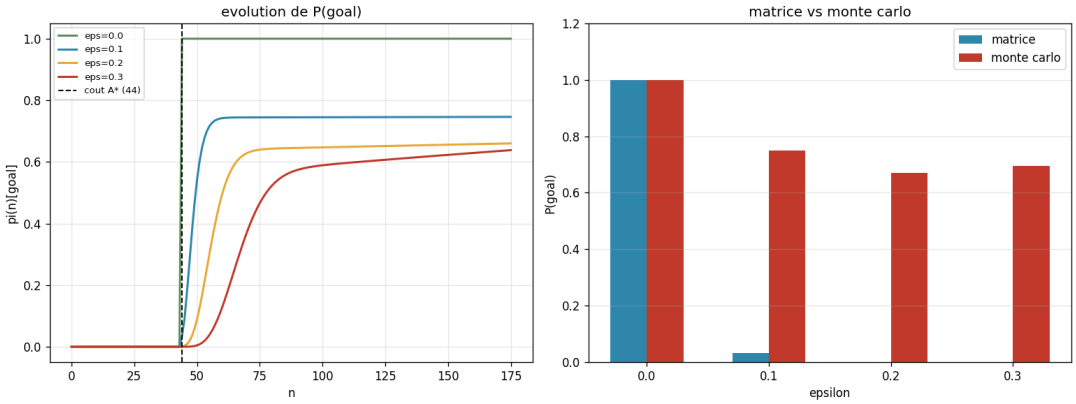

Figure 5: Impact de ϵ sur $\pi^{(n)}[GOAL]$ et comparaison matrice vs Monte-Carlo.

4.3 Classes de communication et absorption

L'algorithme de Kosaraju sur le graphe de transitions (avec $\epsilon > 0$) révèle : **32 classes** au total, dont **31 transitoires** et **1 seule récurrente** : l'état GOAL. Tous les états transitoires sont absorbés vers GOAL avec probabilité 1.

La matrice fondamentale $N = (I - Q)^{-1}$ donne le temps moyen d'absorption. On observe un phénomène contre-intuitif : pour $\epsilon = 0.1$ le temps théorique monte à **4147 étapes**, tandis que pour $\epsilon = 0.2$ il chute à **782 étapes**. Un faible bruit piège l'agent dans des boucles locales ; un bruit plus élevé agit comme mécanisme d'exploration qui en sort.

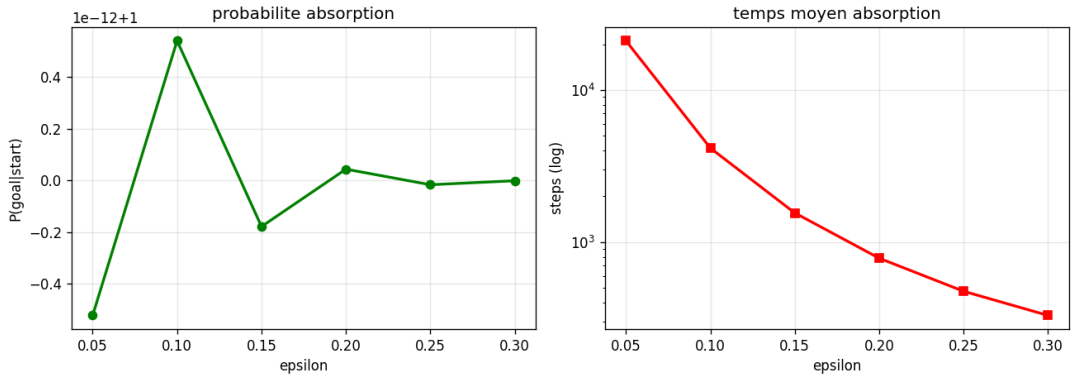

Figure 6: Probabilité d'absorption et temps moyen théorique en fonction de ϵ.

5 Validation Empirique — Simulation Monte-Carlo

10 000 trajectoires sont simulées sur la grille Medium depuis $(0, 0)$, avec une limite de 352 pas. Les résultats confirment le modèle théorique :

- $\epsilon = 0.0$: 100% de réussite, médiane = moyenne = 44 étapes.
- $\epsilon = 0.1$: 74.4% de réussite, moyenne 49.8 étapes.
- $\epsilon = 0.2$ / 0.3 : 68–72% de réussite, mais la moyenne explose (66.5 / 96.7 étapes).

L'écart médiane/moyenne révèle une **distribution à longue traîne** : la majorité des agents arrivent proche de la médiane, mais une minorité tombe dans des boucles locales qui tirent la moyenne vers le haut (pour $\epsilon = 0.3$: médiane 70, moyenne 96.7).

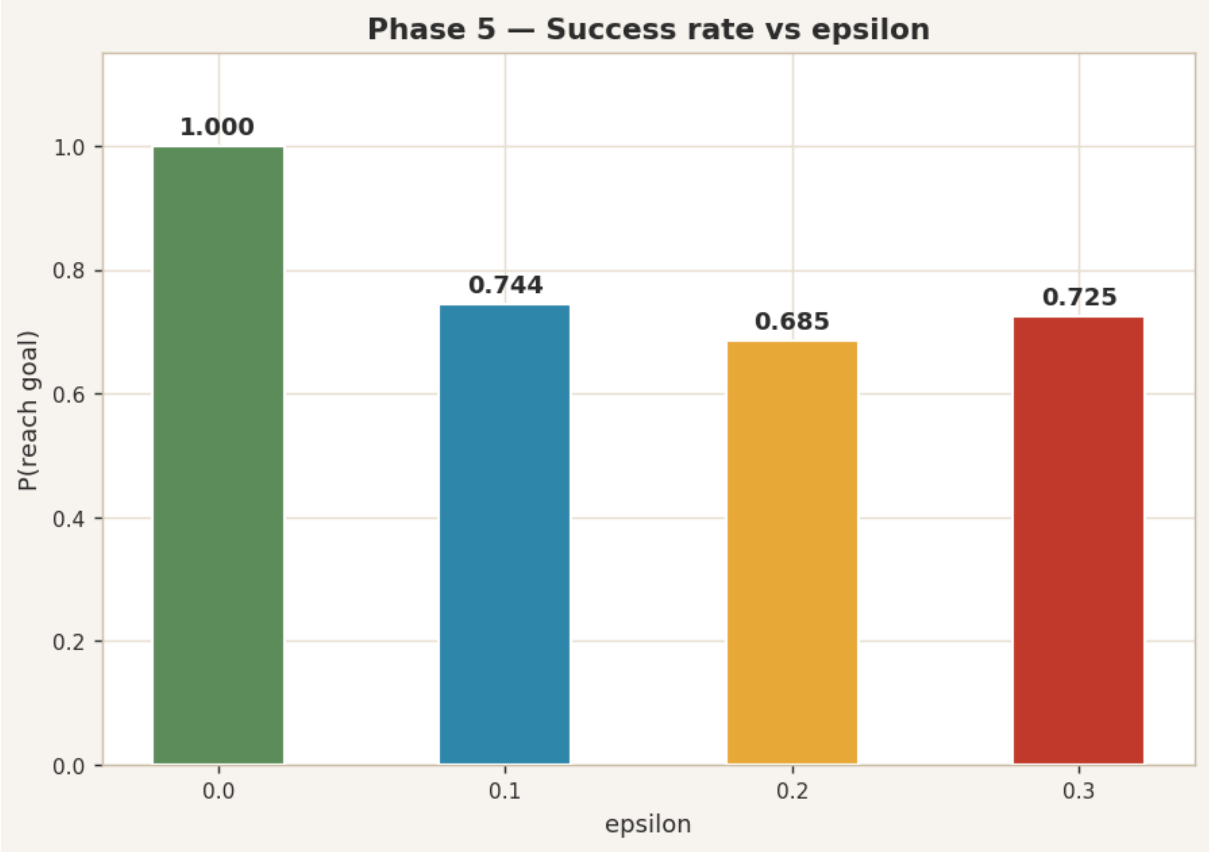

Figure 7: Taux de réussite empirique en fonction de ϵ (10 000 simulations).

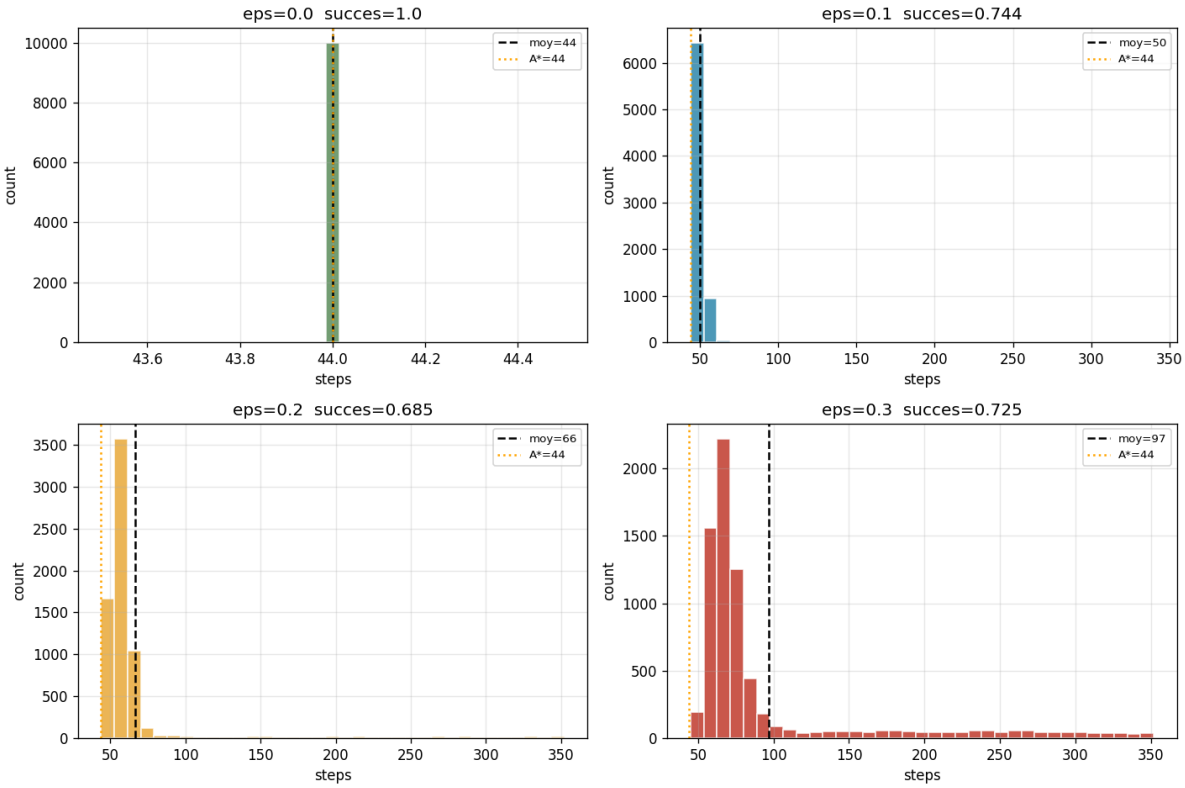

Figure 8: Distribution des temps d'atteinte du but pour chaque valeur de ϵ.

6 Conclusion, Limites et Perspectives

6.1 Synthèse des résultats

Ce projet a démontré que la planification optimale et la robustesse stochastique sont deux objectifs distincts. A^* avec heuristique de Manhattan trouve systématiquement le chemin optimal tout en réduisant l'exploration de 13 à 25% par rapport à UCS. La variante Weighted A^* permet de sacrifier légèrement l'optimalité pour gagner en vitesse. Côté Markov, dès $\epsilon = 0.1$, la probabilité d'exécuter le plan exact s'effondre à 3.34%, et la distribution des temps présente une longue traîne prononcée — phénomène que seule la simulation Monte-Carlo permet de visualiser concrètement.

6.2 Limites du modèle

- **Politique de repli glouton :** les états hors-chemin reçoivent une action qui ignore les obstacles, créant des boucles avec ϵ faible.
- **Passage à l'échelle :** la matrice P de taille $|S|^2$ devient impraticable sur de grandes grilles.
- **Incertitude homogène :** supposer ϵ constant ignore la topologie locale (couloirs étroits vs espaces ouverts).

6.3 Pistes d'amélioration

- **MDP + Value Iteration / Q-Learning :** intégrer nativement l'incertitude dans la politique optimale.

- **Replanification dynamique :** relancer A^* dès que l'agent quitte le chemin prévu.
- **IDA* / SMA* :** maintenir l'optimalité sur de grandes grilles sans saturer la mémoire.